Table des matières

Labo 07 — Charger en incrémental 1
Avant de commencer 3
Étape 1 — Trois mois arrivent 3
Étape 2 — Ce qu'un lot change sans qu'on demande rien 4
Étape 3 — La télémétrie, telle qu'elle arrive 5
Étape 4 — `stg_sorties` 7
Étape 5 — `fct_sorties`, d'abord en table 8
Étape 6 — Le rendre incrémental 9
Étape 7 — Les deux autres mois 10
Étape 8 — Ce que le filtre naïf a perdu 11
Étape 9 — Le filtre qui tient 12
Étape 10 — La fenêtre de reprise 12
Étape 11 — L'idempotence 13
Étape 12 — Le témoin de la télémétrie 14
Étape 13 — Deux environnements 15
Étape 14 — Le contrôle qui compte : tranches contre bloc 16
Étape 15 — La dimension des codes d'anomalie 16
Étape 16 — `fct_codes_sortie`, et la seconde livraison 17
Étape 17 — Ce que la dimension fait voir 19
Étape 18 — Le verdict 19
Étape 19 — La livraison 21
Récapitulatif 22
Dépannage 22

Labo 07 — Charger en incrémental

© 2026 iAlexMG. Tous droits réservés.

Labo · Cours 7/9 — Formation Analytics Engineering

Ce que vous allez faire. Faire entrer trois mois de données dans un entrepôt qui n'en attendait plus, et voir ce que cela change partout. Vous chargez la télémétrie, la dernière source du parcours et la plus difficile : du JSON imbriqué, une liste, un champ qui change de nom au milieu de l'histoire. Vous en tirez un sixième fait, vous le rendez incrémental avec le filtre que tout le monde écrit, et vous mesurez ce qu'il perd. Puis vous le corrigez. Objectif de fin : l'incident du compresseur d'air, que le Cours 06 avait laissé sans verdict, en a un.

Deux terminaux servent dans ce labo. L'Anaconda Prompt, environnement `formations` actif, sert à dbt, à Git et au pipeline de Data Engineering ; il s'ouvre tantôt dans `Projet_Boreal_intermediaire/01_de`, tantôt dans `Projet_Boreal_intermediaire/02_ae`, et chaque étape dit lequel. La ligne de commande DuckDB sert à vérifier ce que dbt a écrit, et elle doit être fermée quand dbt ou le pipeline travaillent.

La liste de contrôle du cours :

- ☐ Les trois mois de 2026 générés, et le lot de janvier chargé
- ☐ Ce qu'un lot change aux cinq faits existants, chiffré
- ☐ Les 154 réclamations réécrites, repérées
- ☐ La télémétrie brute constatée : doublons, dérive de schéma, retards
- ☐ `stg_sorties` : 171 699 sorties, les deux noms de la température réunis
- ☐ `fct_sorties` en table : 171 699 lignes
- ☐ Le fait rendu incrémental, avec le filtre naïf
- ☐ Les lots de février et de mars chargés
- ☐ La perte du filtre naïf mesurée : 47 sorties
- ☐ Le filtre corrigé, la fenêtre de reprise posée, `--full-refresh` : 201 542
- ☐ L'idempotence : deux passes de suite, aucun changement
- ☐ Le témoin du compteur d'heures moteur, au vert
- ☐ Les cibles `dev` et `prod` séparées
- ☐ `dim_anomalie` promue, depuis un `seed`
- ☐ `fct_codes_sortie` : 1 155 codes levés en roulant

© 2026 iAlexMG. Tous droits réservés.

- ☐ La seconde livraison de Data Engineering reprise, l'usine à 4 368 heures
- ☐ Le verdict de l'incident, lu dans deux sources indépendantes
- ☐ `LIVRAISON.md` complété

Le labo se fait en deux séances. La première va jusqu'à l'étape 11, la seconde porte les dimensions, les environnements et le verdict.

Avant de commencer

Ce labo part du projet tel que le Cours 06 l'a laissé. Vérifiez-le dans l'Anaconda Prompt, depuis `Projet_Boreal_intermediaire/02_ae` :

```
dbt build --exclude snp_concessionnaire
```

`dbt build` doit rendre `PASS=103 WARN=0 ERROR=0` sur 104 nœuds. Notez les cinq comptes de départ, vous en aurez besoin dans deux étapes :

```
select
    (select count(*) from marts.fct_ventes) as ventes,
    (select round(sum(valeur_nette), 2) from marts.fct_ventes) as valeur_nette,
    (select count(*) from marts.fct_reclamations) as reclamations,
    (select count(*) from marts.fct_enregistrements) as enregistrements,
    (select count(*) from marts.fct_production) as production;
```

Attendu : 75 652 lignes de vente pour 430 148 012,16 $, 3 348 réclamations, 26 732 enregistrements et 27 775 véhicules assemblés. Fermez ensuite la ligne de commande DuckDB, par `.exit`.

Étape 1 — Trois mois arrivent

Théorie : `Theorie_07_AE.md`, section 1.

Les sources de Boréal vont jusqu'au 31 décembre 2025 parce que c'est ainsi que vous les avez générées au Cours 00. Générez-les maintenant jusqu'à la fin du premier trimestre de 2026, depuis le dossier `Data_Domain` du dépôt des formations :

```
python _univers/boreal/generer_boreal.py --projet Projet_Boreal_intermediaire --jusqua 2026-03
```

Comptez 80 secondes. Le générateur annonce 1 989 fichiers et 177,5 Mo, contre 1 763 et 138,4 Mo auparavant, et il confirme que sa sortie est conforme à la référence des cours.

© 2026 iAlexMG. Tous droits réservés.

Rien n'est encore dans l'entrepôt : ce sont des fichiers sur le disque, exactement comme ceux que l'ERP dépose le premier de chaque mois. Les faire entrer est le travail de Data Engineering, et sa porte d'entrée accepte une fenêtre. Depuis `Projet_Boreal_intermediaire/01_de`, chargez le seul mois de janvier :

```
python -m pipeline raw --depuis 2026-01 --jusqua 2026-01
```

Attendu : 47 fichiers, 23 387 lignes, dont 31 fichiers et 16 076 messages pour la télémétrie et un extrait de 595 lignes pour `VBAK`. Les exports qui n'ont pas changé, les actifs et la météo, affichent « à jour » et ne sont pas relus : le chargeur les reconnaît à leur empreinte.

Rappel — vous n'écrivez pas ce chargement, et c'est voulu. L'ingestion appartient à Data Engineering, qui en a fait son Cours 06. Ce qui vous revient commence à la `RAW`, et la seule chose que vous ayez à savoir d'un lot est qu'il est arrivé.

Étape 2 — Ce qu'un lot change sans qu'on demande rien

Théorie : `Theorie_07_AE.md`, sections 1 et 7.

Vous n'avez touché à aucun modèle. Reconstruisez quand même, depuis `Projet_Boreal_intermediaire/02_ae` :

```
dbt build --exclude snp_concessionnaire
```

Puis reprenez la requête de la page précédente, dans la ligne de commande DuckDB.

Le fait	Au 31 décembre	Après le lot de janvier
`fct_ventes`	75 652 lignes, 430 148 012,16 $	77 324 lignes, 443 242 479,39 $
`fct_enregistrements`	26 732	27 276
`fct_reclamations`	3 348	3 646
`fct_production`	27 775	28 302
`fct_disponibilite_actifs`	22 090	22 090

Quatre faits sur cinq grossissent, et le cinquième ne bouge pas d'une ligne. Ce n'est pas une erreur : l'instantané de l'usine lit le schéma `argent`, qui est la seconde livraison de Data Engineering et qui n'a pas été rafraîchie. Vous vous en occuperez à l'étape 16, et l'écart vaut qu'on le remarque maintenant : deux livraisons du même fournisseur peuvent ne pas marcher du même pas.

Le fait des réclamations mérite un regard de plus, parce qu'il n'a pas seulement grossi. Comparez l'état courant avant et après le lot :

```
with avant as (
    select objet from (
        select objet, row_number() over (
            partition by objet ->> 'reclamation_id' order by _lot desc) as rang
```


© 2026 iAlexMG. Tous droits réservés.

```
        from raw.garanties_reclamations where _lot < '2026-01'
    ) where rang = 1
),
apres as (
    select objet from (
        select objet, row_number() over (
            partition by objet ->> 'reclamation_id' order by _lot desc) as rang
        from raw.garanties_reclamations
    ) where rang = 1
)
select count(*) as deja_ecrites,
       count(*) filter (where a.objet::varchar is distinct from b.objet::varchar) as changees
from avant as a
inner join apres as b on (b.objet ->> 'reclamation_id') = (a.objet ->> 'reclamation_id');
```

Attendu : 3 348 déjà écrites, 154 changées. Ce sont des dossiers ouverts avant Noël qui ont reçu leur décision ou leur paiement pendant l'hiver, et ils expliquent à eux seuls pourquoi le fait d'accumulation restera en `table` jusqu'à la fin de ce cours.

Renvoi : `is distinct from`, la comparaison qui compte les `NULL`, est au Cours 01 de SQL, section 8.

À retenir — le neuf n'est pas seulement ce qui s'ajoute. Sur ce lot, 298 réclamations sont nouvelles et 154 sont des corrections de lignes existantes. Un pipeline qui ne connaîtrait que la première catégorie serait faux de 4,6 pour cent sans que rien ne l'indique.

Étape 3 — La télémétrie, telle qu'elle arrive

Théorie : `Theorie_07_AE.md`, section 2.

Avant d'écrire une ligne de modèle, regardez la source. Dans la ligne de commande DuckDB :

```
describe raw.telemetrie_sorties;
select * from raw.telemetrie_sorties limit 1;
```

Sept colonnes, dont une seule porte la donnée : `message`, de type `JSON`. À côté, `recu_le` dit quand l'abonné a reçu le message, `qos` la qualité de service de la publication, et les trois colonnes techniques de la `RAW` disent d'où vient la ligne.

Trois comportements sont à constater, et chacun décidera d'une ligne du modèle.

Les doublons, d'abord. Le courtier livre « au moins une fois », ce que `qos` vaut 1 annonce :

```
select count(*) as lignes,
       count(distinct message ->> 'message_id') as identifiants,
       count(*) - count(distinct message ->> 'message_id') as doublons
from raw.telemetrie_sorties;
```


© 2026 iAlexMG. Tous droits réservés.

Attendu, avec le lot de janvier chargé : 174 222 lignes pour 171 699 identifiants, donc 2 523 doublons. Vérifiez ensuite qu'un doublon arrive toujours le même jour que son original, parce que c'est une propriété dont vous vous servirez à l'étape 10 :

```
with d as (
    select message ->> 'message_id' as id,
           min(recu_le::timestamptz) as premier,
           max(recu_le::timestamptz) as dernier
    from raw.telemetrie_sorties
    group by 1 having count(*) > 1
)
select count(*) as en_double,
       count(*) filter (where premier::date = dernier::date) as le_meme_jour,
       max(datediff('minute', premier, dernier)) as ecart_max_minutes
from d;
```

Attendu : tous le même jour, et une minute d'écart au plus.

La dérive de schéma, ensuite. Le micrologiciel 2.0 du module télématique a été diffusé à partir du 1er mai 2025 :

```
select message ->> 'micrologiciel' as micrologiciel,
       count(*) as messages,
       count(*) filter (where (message -> 'temp_moteur_max') is not null) as ancien_nom,
       count(*) filter (where (message -> 'temperature_moteur_max') is not null) as nouveau_nom,
       count(*) filter (where (message -> 'heures_moteur') is not null) as heures_moteur
from raw.telemetrie_sorties
group by 1 order by 1;
```

Attendu : 71 029 messages en `1.3.0`, 17 649 en `1.6.2`, 85 544 en `2.0.1`. Trois versions, et une coupure nette : les deux premières portent `temp_moteur_max`, la troisième porte `temperature_moteur_max` et ajoute un compteur d'heures moteur. Aucun message ne porte les deux noms.

Piège — les parenthèses autour d'un `->` ne sont pas décoratives. Le Cours 06 l'a rencontré sur `IS NULL`, et cela vaut aussi pour `LIKE` : sans parenthèses, DuckDB lit `message ->> ('micrologiciel' like '2.%')` et se plaint d'une conversion impossible. Le message d'erreur ne nomme jamais la vraie cause.

Les retards, enfin, qui sont le sujet de la seconde moitié de ce labo :

```
select count(*) as en_retard,
       max(datediff('hour', (message ->> 'fin')::timestamptz, recu_le::timestamptz)) as retard_max_h
from raw.telemetrie_sorties
where recu_le::timestamptz - (message ->> 'fin')::timestamptz > interval 1 hour;
```

Attendu : 6 844 messages en retard, et un retard maximal de 96 heures. Un module hors de la couverture cellulaire garde ses messages et les publie au retour, et le phénomène suit la saison : février, janvier et décembre en comptent le plus, ce qui est exactement ce qu'on attend d'un parc de motoneiges au fond d'une vallée.

© 2026 iAlexMG. Tous droits réservés.

Étape 4 — `stg_sorties`

Théorie : `Theorie_07_AE.md`, section 2.

Déclarez d'abord la source, dans `models/staging/_sources.yml`, au bloc `raw` :

```yaml
      - name: telemetrie_sorties
        description: "Un message par sortie d'un véhicule connecté, objet JSON."
```

Puis écrivez `models/staging/stg_sorties.sql`. L'ordre des deux morceaux compte, et c'est le même qu'au Cours 06 : on dédoublonne d'abord, on aplatit ensuite.

```sql
with dedoublonne as (

    select
        message,
        recu_le,
        _lot,
        _charge_le,
        row_number() over (
            partition by message ->> 'message_id'
            order by recu_le
        ) as rang

    from {{ source('raw', 'telemetrie_sorties') }}

)

select
    message ->> 'message_id' as numero_message,
    message ->> 'sernr' as numero_serie,
    message ->> 'micrologiciel' as micrologiciel,
    (message ->> 'debut')::timestamptz as debut,
    (message ->> 'fin')::timestamptz as fin,
    recu_le::timestamptz as recu_le,
    (message ->> 'distance_km')::decimal(18, 2) as distance_km,
    (message ->> 'heures_moteur')::decimal(18, 1) as heures_moteur,
    (message ->> 'regime_moyen')::integer as regime_moyen,
    (message ->> 'regime_max')::integer as regime_max,
    coalesce(
        (message ->> 'temperature_moteur_max')::decimal(18, 1),
        (message ->> 'temp_moteur_max')::decimal(18, 1)
    ) as temperature_moteur_max,
```


© 2026 iAlexMG. Tous droits réservés.

```
    (message ->> 'temp_exterieure')::decimal(18, 1) as temperature_exterieure,
    (message ->> 'tension_batterie_min')::decimal(18, 2) as tension_batterie_min,
    (message -> 'position' ->> 'lat')::decimal(18, 2) as latitude,
    (message -> 'position' ->> 'lon')::decimal(18, 2) as longitude,
    message -> 'codes' as codes,
    _lot as lot_source,
    _charge_le as charge_le

from dedoublonne
where rang = 1
```

Quatre lignes méritent un mot.

Le `coalesce` est la réunion des deux noms du champ, et c'est la seule ligne du projet qui sache qu'il y a eu deux versions de micrologiciel. Il tient dans cet ordre-là parce que le nouveau nom est celui qui durera.

`position` se descend en deux crans, `->` puis `->>`, le premier rendant du JSON et le second du texte.

`codes` reste en JSON, comme l'historique des réclamations au Cours 06. Une liste n'est pas une colonne : elle se dépliera à l'étape 15, dans son propre fait.

`recu_le` et `_charge_le` sont deux horloges, et elles ne disent pas la même chose. La première est l'arrivée du message chez Boréal, la seconde l'instant où Data Engineering l'a écrit dans la `RAW`. Gardez les deux : l'étape 9 vous dira à quoi elles servent.

Construisez, depuis `Projet_Boreal_intermediaire/02_ae` :

```
dbt run --select stg_sorties
```

Vérifiez dans la ligne de commande DuckDB, après avoir fermé dbt :

```
select count(*) as sorties, count(distinct numero_serie) as vehicules,
       min(fin)::date as du, max(fin)::date as au
from staging.stg_sorties;
```

Attendu : 171 699 sorties pour 7 209 véhicules, du 8 septembre 2023 au 31 janvier 2026. Les 2 523 doublons ont disparu, et aucun message n'a été perdu au passage.

Étape 5 — `fct_sorties`, d'abord en table

Théorie : `Theorie_07_AE.md`, section 3.

Le sixième fait est un fait de transaction, la forme la plus simple, et vous en avez déjà écrit deux. Une ligne par sortie, deux clés, six mesures. Dans `models/marts/fct_sorties.sql` :

© 2026 iAlexMG. Tous droits réservés.

```
select
    (s.fin at time zone 'America/Toronto')::date as cle_date,
    {{ dbt_utils.generate_surrogate_key(['s.numero_serie']) }} as cle_vehicule,
    s.numero_message,
    s.numero_serie,
    s.micrologiciel,
    s.distance_km,
    datediff('minute', s.debut, s.fin) as minutes_en_sortie,
    s.regime_moyen,
    s.regime_max,
    s.temperature_moteur_max,
    s.temperature_exterieure,
    s.tension_batterie_min,
    json_array_length(s.codes) as codes_leves,
    s.recu_le,
    s.charge_le

from {{ ref('stg_sorties') }} as s
```

`at time zone` convertit un instant absolu en heure locale avant d'en prendre la date, comme au Cours 06. Ce n'est pas une précaution de style : sur cette source, 34 558 sorties sur 201 542 changent de jour selon qu'on les lit en heure locale ou en temps universel, parce qu'une sortie finit souvent le soir.

`codes_leves` compte la liste sans la déplier, ce qui est la mesure qui a sa place au grain de la sortie.

Construisez, en chronométrant :

```
dbt run --select fct_sorties
```

Attendu : le modèle se bâtit en moins d'une seconde. Vérifiez :

```
select count(*) as sorties, round(sum(distance_km), 1) as km,
       round(avg(distance_km), 2) as km_moyens,
       round(avg(minutes_en_sortie), 1) as minutes_moyennes,
       sum(codes_leves) as codes_leves
from marts.fct_sorties;
```

Attendu : 171 699 sorties, 6 563 245,0 kilomètres, 38,23 kilomètres et 103,4 minutes en moyenne, 946 codes levés. Ce sont les chiffres de l'entrepôt au 31 janvier 2026 ; ils bougeront deux fois d'ici la fin du labo.

Étape 6 — Le rendre incrémental

Théorie : `Theorie_07_AE.md`, section 3.

© 2026 iAlexMG. Tous droits réservés.

Ajoutez un bloc de configuration en tête du modèle, et un bloc conditionnel à la fin :

```
{{
    config(
        materialized = 'incremental',
        unique_key = 'numero_message',
        incremental_strategy = 'delete+insert'
    )
}}

{% if is_incremental() %}
where (s.fin at time zone 'America/Toronto')::date > (select max(cle_date) from {{ this }})
{% endif %}
```

C'est le filtre que tout le monde écrit, et c'est celui que vous allez démonter. Écrivez-le tel quel : une faute qu'on a faite soi-même ne se réapprend pas.

```
dbt run --select fct_sorties
```

La table existe déjà, donc `is_incremental()` vaut vrai et le filtre s'applique. Comme rien n'est arrivé depuis, il ne laisse rien passer, et le compte ne bouge pas : 171 699.

Piège — la bascule d'une `table` vers `incremental` ne reconstruit rien. dbt voit une table qui existe et se met tout de suite en régime incrémental. Si la table portait des lignes fausses, elles y restent. Après toute bascule, un `--full-refresh` est la seule façon de repartir propre, et l'étape 10 le fera.

Étape 7 — Les deux autres mois

Théorie : `Theorie_07_AE.md`, section 1.

Chargez février, puis reconstruisez ; chargez mars, puis reconstruisez. Depuis `Projet_Boreal_intermediaire/01_de` pour la première commande, `Projet_Boreal_intermediaire/02_ae` pour la seconde, et la ligne de commande DuckDB fermée à chaque fois :

```
python -m pipeline raw --depuis 2026-02 --jusqua 2026-02

dbt build --exclude snp_concessionnaire
```

Puis la même paire pour `2026-03`. Les trois lots de 2026 pèsent 47, 42 et 45 fichiers, pour 23 387, 23 655 et 20 820 lignes.

Voici les faits, passe par passe. Recopiez ce tableau à mesure : c'est le journal d'un pipeline en régime permanent, et c'est la première fois du parcours que vous en tenez un.

La passe	fct_ventes	Valeur nette	fct_reclamations	fct_production	fct_sorties
Au 2025-12	75 652	430 148 012,16 $	3 348	27 775	—

© 2026 iAlexMG. Tous droits réservés.

La passe	fct_ventes	Valeur nette	fct_reclamations	fct_production	fct_sorties
Lot 2026-01	77 324	443 242 479,39 $	3 646	28 302	171 699
Lot 2026-02	78 934	455 459 366,68 $	4 036	29 071	188 600
Lot 2026-03	80 618	465 924 613,14 $	4 337	29 782	201 495

Tout est au vert, `PASS=105 WARN=0 ERROR=0` sur 106 nœuds à chaque passe. Notez la dernière colonne : elle sera fausse, et rien ici ne le dit.

Étape 8 — Ce que le filtre naïf a perdu

Théorie : `Theorie_07_AE.md`, section 4.

Le fait est bâti par tranches ; le `STAGING`, lui, est une vue qui voit toute la source. Comparer les deux est donc le contrôle qui coûte le moins cher, et il faut le faire avant de croire un chiffre :

```
select (select count(*) from staging.stg_sorties) as dans_la_source,
       (select count(*) from marts.fct_sorties) as dans_le_fait,
       (select count(*) from staging.stg_sorties)
     - (select count(*) from marts.fct_sorties) as manquantes;
```

Attendu : 201 542, 201 495, et 47 manquantes. Cherchez-les ; `not exists` garde les sorties que le fait ne connaît pas, l'anti-jointure du Cours 07 de SQL, section 4 :

```
select (v.fin at time zone 'America/Toronto')::date as jour_de_sortie,
       count(*) as manquantes,
       min(v.recu_le)::date as recues_le
from staging.stg_sorties as v
where not exists (
    select 1 from marts.fct_sorties as f where f.numero_message = v.numero_message
)
group by 1 order by 1;
```

Jour de sortie	Manquantes	Reçues le
2026-01-30	2	2026-02-01
2026-01-31	22	2026-02-01
2026-02-27	2	2026-03-01
2026-02-28	21	2026-03-01

© 2026 iAlexMG. Tous droits réservés.

Le motif est net : toujours les deux derniers jours de la période, toujours des messages reçus après la coupure. Ce sont les retardataires de l'étape 3, et le filtre les a écartés parce que leur date de sortie était antérieure au maximum déjà chargé. Aucune passe ne les rattrapera, ce maximum ne faisant que monter.

Ce qu'ils emportent :

```
select count(*) as manquantes,
       round(sum(v.distance_km), 1) as km,
       sum(json_array_length(v.codes)) as codes
from staging.stg_sorties as v
where not exists (
    select 1 from marts.fct_sorties as f where f.numero_message = v.numero_message
);
```

Attendu : 47 sorties, 2 558,2 kilomètres et 2 codes d'anomalie. Deux centièmes de pour cent, et c'est le plus inquiétant : une erreur assez petite pour qu'aucun contrôle de bon sens ne la voie, et assez systématique pour fausser tous les rapports de fin de mois. Deux passes ont suffi à la produire ; il y en aurait douze dans une année, et trois cent soixante-cinq si le pipeline suivait la cadence de la source.

À retenir — le filtre incrémental n'a pas de filet. Un test de grain ne le voit pas, un test de relation non plus, et la réconciliation des ventes n'a rien à dire sur la télémétrie. La seule chose qui attrape ce défaut est la comparaison avec la source, et elle ne se fait pas toute seule.

Étape 9 — Le filtre qui tient

Théorie : `Theorie_07_AE.md`, section 4.

Remplacez le filtre par celui-ci :

```
{% if is_incremental() %}
where s.recu_le > (select max(recu_le) from {{ this }})
{% endif %}
```

Un retardataire garde une date de sortie ancienne, mais sa date de réception est celle du jour où il est arrivé : ce filtre le prend. C'est la seule différence entre les deux versions, et c'est toute la leçon du cours.

Ne reconstruisez pas encore : l'étape suivante ajoute la fenêtre, et un seul `--full-refresh` servira pour les deux.

Étape 10 — La fenêtre de reprise

Théorie : `Theorie_07_AE.md`, section 5.

Le filtre de l'étape 9 est juste, et il repose sur une hypothèse que vous avez vérifiée à l'étape 3 : un message n'arrive jamais avec une date de réception antérieure à une date déjà vue, puisque les doublons tombent le même jour à une minute près. Rien n'impose cette propriété, et c'est exactement ce qu'une fenêtre de reprise assure.

© 2026 iAlexMG. Tous droits réservés.

```
{% if is_incremental() %}
where s.recu_le > (
    select max(recu_le) - interval '{{ var("fenetre_de_reprise_jours", 3) }}' day
    from {{ this }}
)
{% endif %}
```

Trois jours, et la largeur se justifie : le plus long retard mesuré sur cette source est de 96 heures. Le `var()` avec sa valeur par défaut permet d'élargir la fenêtre sans toucher au modèle, `--vars "{fenetre_de_reprise_jours: 7}"`, ce qui est la première chose qu'on fait après un incident.

Reconstruisez de zéro, puisque le filtre a changé :

```
dbt run --select fct_sorties --full-refresh
```

Puis vérifiez que la comparaison de l'étape 8 ne trouve plus rien :

```
select count(*) as sorties, count(distinct numero_message) as messages,
       count(distinct numero_serie) as vehicules,
       round(sum(distance_km), 1) as km,
       max(cle_date) as derniere_sortie
from marts.fct_sorties;
```

Attendu : 201 542 sorties pour autant de messages distincts, 7 644 véhicules, 7 965 681,0 km, et une dernière sortie le 31 mars 2026. Les 47 manquantes sont revenues, et rien n'a été compté deux fois.

Étape 11 — L'idempotence

Théorie : `Theorie_07_AE.md`, sections 1 et 6.

Une passe rejouée ne doit rien changer. Lancez deux fois de suite, sans rien charger entre les deux :

```
dbt run --select fct_sorties
dbt run --select fct_sorties
```

Puis :

```
select count(*) as sorties, count(distinct numero_message) as messages
from marts.fct_sorties;
```

Attendu : 201 542 et 201 542, inchangés. Les deux passes ont relu la fenêtre de trois jours et réécrit ces lignes, et `unique_key` a fait que la réécriture remplace au lieu d'ajouter.

Retirez maintenant la ligne `unique_key` du bloc de configuration, relancez une fois, et regardez :

© 2026 iAlexMG. Tous droits réservés.

```
select count(*) as lignes, count(distinct numero_message) as messages
from marts.fct_sorties;
```

Attendu : 202 330 lignes pour 201 542 messages distincts, soit 788 doublons, qui sont exactement les lignes que la fenêtre de trois jours a relues. Remettez la ligne, relancez avec `--full-refresh`, et retenez ce que vous venez de voir : le filtre sert la vitesse, la clé sert la justesse, et c'est la clé qui manque dans les incidents qu'on raconte.

Astuce — vérifiez ce qu'on vous dit d'un outil. « Un full-refresh détruit un snapshot » est une phrase qu'on entend souvent, et elle est fausse sur cette pile : `dbt snapshot --full-refresh` est refusé par dbt 1.12.3, qui ne connaît pas l'option sur cette commande, et `dbt build --full-refresh` laisse les 205 versions et les 49 fermetures intactes. Lancez les deux si vous voulez le voir. Ce qui demande de l'ordre, pour un instantané, c'est la façon dont son histoire se monte, et le Cours 05 l'a posée.

Étape 12 — Le témoin de la télémétrie

Théorie : `Theorie_07_AE.md`, section 3.

Chaque étoile du parcours a son témoin, et celle-ci n'a aucun total indépendant à retrouver : personne ne tient ailleurs le compte des kilomètres parcourus par les clients. Comme à l'usine au Cours 06, le témoin se fabrique donc dans la source, et le micrologiciel 2.0 en fournit un. Il publie un compteur d'heures moteur cumulatif : d'une sortie à la suivante du même véhicule, sa hausse doit valoir la durée de la sortie d'arrivée.

Dans `tests/assert_compteur_heures_moteur.sql` :

```
with suite as (

    select
        numero_serie,
        fin,
        heures_moteur,
        datediff('minute', debut, fin) / 60.0 as duree_h,
        lag(heures_moteur) over (partition by numero_serie order by fin) as heures_avant

    from {{ ref('stg_sorties') }}
    where heures_moteur is not null

)

select numero_serie, fin, heures_avant, heures_moteur, duree_h
from suite
where heures_avant is not null
  and abs((heures_moteur - heures_avant) - duree_h) > 0.5
```

```
dbt test --select assert_compteur_heures_moteur
```


© 2026 iAlexMG. Tous droits réservés.

Attendu : au vert. Mesurez ensuite ce que le test vérifie vraiment, parce qu'un test au vert ne dit pas de combien il l'est :

```
with suite as (
    select numero_serie, heures_moteur,
           datediff('minute', debut, fin) / 60.0 as duree_h,
           lag(heures_moteur) over (partition by numero_serie order by fin) as avant
    from staging.stg_sorties where heures_moteur is not null
)
select count(*) as couples, count(distinct numero_serie) as vehicules,
       count(*) filter (where heures_moteur < avant) as reculs,
       round(avg(abs((heures_moteur - avant) - duree_h)), 3) as ecart_moyen_h,
       round(max(abs((heures_moteur - avant) - duree_h)), 3) as ecart_max_h
from suite where avant is not null;
```

Attendu : 106 636 couples sur 7 211 véhicules, aucun recul du compteur, un écart moyen de 0,032 heure et un écart maximal de 0,1 heure. La tolérance du test est d'une demi-heure, cinq fois le pire écart observé, et c'est délibéré : on cherche une dérive, pas un arrondi.

Piège — comparer les totaux plutôt que les écarts. La tentation est de vérifier le compteur d'un coup, par `max(heures_moteur) - min(heures_moteur)` contre la somme des durées. Mesuré, l'entrepôt rend 183 527,2 heures d'un côté et 195 898,7 de l'autre, et on croit tenir un défaut. Il n'y en a pas : sur *n* sorties, la différence des extrêmes ne couvre que *n − 1* intervalles, et les 12 371,5 heures manquantes valent exactement une sortie moyenne par véhicule.

Étape 13 — Deux environnements

Théorie : `Theorie_07_AE.md`, section 9.

Ajoutez une cible de production à votre `profiles.yml`, sous `outputs:`, à côté de `dev` :

```
    prod:
      type: duckdb
      path: C:/Users/<vous>/.../Data_Domain/Projet_Boreal_intermediaire/plateforme/entrepot/boreal.duckdb
      schema: staging
      threads: 4
```

Puis rendez la macro du Cours 03 consciente de l'environnement, dans `macros/generate_schema_name.sql` :

```
{% macro generate_schema_name(custom_schema_name, node) -%}
    {%- set nom = custom_schema_name | default(target.schema, true) | trim -%}
    {%- if target.name == 'prod' -%}
        {{ nom }}
    {%- else -%}
        dev_{{ nom }}
    {%- endif -%}
{%- endmacro %}
```


© 2026 iAlexMG. Tous droits réservés.

C’est la seule retouche de cette macro dans tout le parcours. Constatez l’effet :

```
dbt run --select stg_sorties fct_sorties
```

dbt annonce `dev_staging.stg_sorties` et `dev_marts.fct_sorties`. Relancez sur la production :

```
dbt run --target prod --select stg_sorties fct_sorties
```

Et vérifiez les quatre schémas :

```
select table_schema, count(*) as objets
from information_schema.tables
where table_schema in ('staging', 'marts', 'dev_staging', 'dev_marts')
group by 1 order by 1;
```

Le même code, le même dépôt, deux endroits. À partir d’ici, toutes les commandes de ce labo portent `--target prod`, et vos essais se font sans `--target`.

Astuce — nettoyer les schémas de développement. `dbt run-operation` et les macros de nettoyage viendront au Cours 08 ; pour l’instant, deux `drop schema ... cascade` dans la ligne de commande DuckDB suffisent, et le faire de temps en temps évite de lire un chiffre de `dev_marts` en croyant lire la production.

Étape 14 — Le contrôle qui compte : tranches contre bloc

Théorie : `Theorie_07_AE.md`, section 6.

Un chargement par tranches et un chargement d’un bloc doivent donner la même chose. Vous avez fait le premier, en quatre passes ; faites le second :

```
dbt build --target prod --exclude snp_concessionnaire --full-refresh
```

Puis reprenez la comparaison de l’étape 8. Attendu : 201 542 des deux côtés, aucune manquante, aucun doublon.

C’est le seul contrôle sérieux qu’on ait sur un modèle incrémental, et il doit se refaire après toute retouche du modèle ou de ce qu’il lit. Un incrémental qui ne se compare jamais à son plein rebâti est une promesse que personne n’a vérifiée.

Étape 15 — La dimension des codes d’anomalie

Théorie : `Theorie_07_AE.md`, section 8.

Le Cours 06 a laissé `dim_anomalie` dégénérée, et a annoncé que la question se rouvrirait ici. Elle se rouvre parce qu’un second fait porte le même code : le technicien le lit au calculateur et le reporte dans la réclamation, le module télématique le publie dans son résumé de sortie.

Les libellés ne vivent dans aucune table. Ils sont dans un guide du portail des concessionnaires, `GUI-CODES`, que le corpus livre. C’est le cas d’usage d’un `seed`. Créez `seeds/anomalies.csv` :

© 2026 iAlexMG. Tous droits réservés.

```
code_anomalie,libelle,systeme
D240,patinage de la courroie de transmission,transmission
E120,tension de batterie basse,electrique
H510,pression d'huile basse,moteur
T310,surchauffe du moteur,refroidissement
T320,niveau de liquide de refroidissement bas,refroidissement
```

Déclarez où les seeds atterrissent, dans `dbt_project.yml` :

```
seeds:
  boreal_ae:
    +schema: marts
```

Puis chargez-les :

```
dbt seed --target prod
```

Et écrivez `models/marts/dim_anomalie.sql`, qui n'a rien d'autre à faire que de nommer le seed comme une source du projet :

```
select
    code_anomalie,
    libelle,
    systeme

from {{ ref('anomalies') }}
```

Un `seed` se référence par `ref()`, comme un modèle, et c'est ce qui le place dans le graphe. Un modèle par-dessus plutôt qu'un usage direct du seed coûte quatre lignes et rapporte deux choses : la dimension porte un nom de dimension, et le jour où un système livrera enfin ce référentiel, une seule ligne changera.

Étape 16 — `fct_codes_sortie`, et la seconde livraison

Théorie : `Theorie_07_AE.md`, section 8.

Une sortie peut lever plusieurs codes, donc le grain n'est pas celui de `fct_sorties` et la liste demande son propre fait. C'est un fait sans mesure, la forme du Cours 06. Dans `models/marts/fct_codes_sortie.sql` :

```
{{
    config(
        materialized = 'incremental',
        unique_key = ['numero_message', 'code_anomalie'],
        incremental_strategy = 'delete+insert'
    )
}}
```


© 2026 iAlexMG. Tous droits réservés.

```
select
    (s.fin at time zone 'America/Toronto')::date as cle_date,
    {{ dbt_utils.generate_surrogate_key(['s.numero_serie']) }} as cle_vehicule,
    s.numero_message,
    c.value ->> '$' as code_anomalie,
    s.recu_le

from {{ ref('stg_sorties') }} as s,
    json_each(s.codes) as c

{% if is_incremental() %}
where s.recu_le > (
    select max(recu_le) - interval '{{ var("fenetre_de_reprise_jours", 3) }}' day
    from {{ this }}
)
{% endif %}
```

`json_each` déplie la liste en lignes, une par élément, et une liste vide ne produit rien : c'est la jointure latérale que le Cours 08 de SQL avancé enseigne, sous la forme que DuckDB accepte. `unique_key` porte deux colonnes, parce que le grain en a deux.

Il reste la seconde livraison de Data Engineering, celle de l'étape 2. Depuis `Projet_Boreal_intermediaire/01_de`, puis depuis `Projet_Boreal_intermediaire` :

```
python -m pipeline bronze
```

```
duckdb -f 01_de/sql/argent_usine.sql
```

Le bronze remonte à 754 572 lignes, et le schéma `argent` passe de 121 495 à 240 240 lignes horaires, l'historien couvrant maintenant 182 jours au lieu de 92.

Ajoutez les tests des trois nouveaux objets dans `models/marts/_marts.yml`, sur le modèle de ceux du Cours 05 : grain, clés, relations, et un `accepted_values` doublé d'un `not_null` sur le système. Puis reconstruisez tout :

```
dbt build --target prod --exclude snp_concessionnaire
```

Attendu : `PASS=128 WARN=0 ERROR=0` sur 129 nœuds, en cinq secondes. Le projet porte 31 modèles, un seed, 98 tests de données et un test unitaire.

Vérifiez les deux nouveautés :

```
select count(*) as codes_leves, count(distinct numero_message) as sorties
from marts.fct_codes_sortie;

select count(*) as disponibilite, count(distinct heure) as heures,
```


© 2026 iAlexMG. Tous droits réservés.

```
       min(cle_date) as du, max(cle_date) as au
from marts.fct_disponibilite_actifs;
```

Attendu : 1 155 codes levés sur 1 140 sorties, quinze sorties en ayant levé deux ; et l'instantané de l'usine à 43 680 lignes sur 4 368 heures, du 1er octobre 2025 au 31 mars 2026.

Étape 17 — Ce que la dimension fait voir

Théorie : `Theorie_07_AE.md`, section 8.

Une dimension conforme sert à compter ensemble ce que deux faits comptent séparément :

```
select a.code_anomalie, a.libelle,
       count(distinct r.cle_vehicule) as reclames,
       count(distinct c.cle_vehicule) as leves_en_roulant
from marts.dim_anomalie as a
left join marts.fct_reclamations as r on r.code_anomalie = a.code_anomalie
left join marts.fct_codes_sortie as c on c.code_anomalie = a.code_anomalie
group by 1, 2 order by 1;
```

Le code	Réclamés	Levés en roulant
D240	239	500
E120	418	155
H510	192	103
T310	174	96
T320	0	211

La dernière ligne est le résultat, et elle n'existe que parce que la dimension existe. Deux cent onze véhicules ont signalé un niveau de liquide de refroidissement bas en roulant, et pas une réclamation ne porte ce code. Sans `dim_anomalie`, ce zéro n'a nulle part où s'afficher : les deux faits ne connaissent que les codes qu'ils portent, et une absence ne sort d'aucune requête qui ne la cherche pas.

Les deux premières lignes disent autre chose, et il faut résister à la tentation de les interpréter vite : un code levé en roulant et un code réclamé ne mesurent pas la même population. Le premier ne concerne que les véhicules connectés dont le client consent à la transmission, le second tous les véhicules sous garantie.

Étape 18 — Le verdict

Théorie : `Theorie_07_AE.md`, section 11.

© 2026 iAlexMG. Tous droits réservés.

Le Cours 06 s'est arrêté sur une cohorte sans conclusion : 240 motoneiges assemblées pendant que le compresseur d'air numéro 2 dérivait, du 15 au 28 octobre 2025, et un taux de réclamation qui ne se distinguait pas de celui des autres. Reprenez la même requête, inchangée, sur l'entrepôt qui porte maintenant trois mois de plus :

```
create or replace temp view exposees as
select p.cle_vehicule
from marts.fct_production as p
inner join marts.dim_actif as a on a.cle_actif = p.cle_actif
where a.actif_id = 'LMN'
  and p.cle_date_assemblage between date '2025-10-15' and date '2025-10-28';

select
    (select count(*) from exposees) as assemblees,
    (select count(*) from exposees x
        join marts.fct_enregistrements e on e.cle_vehicule = x.cle_vehicule) as vendues,
    (select count(distinct r.cle_vehicule) from exposees x
        join marts.fct_reclamations r on r.cle_vehicule = x.cle_vehicule
        where r.cause = 'REFROIDISSEMENT') as refroidissement;
```

Attendu : 240 assemblées, 237 vendues, 34 avec une réclamation de refroidissement. Comparez au reste des motoneiges de l'année-modèle 2026 :

Le groupe	Vendues	Réclamations de refroidissement	Taux
La cohorte exposée	237	34	14,35 %
Les autres motoneiges 2026	2 498	29	1,16 %

Douze fois plus. Et regardez ce que le même calcul donne toutes causes confondues : 27,0 pour cent contre 19,2. Le signal existe, mais il ne se voit que si l'on regarde la bonne cause, ce qui est la raison d'être d'une dimension de cause.

La télémétrie, qui n'existait pas dans l'entrepôt au Cours 06, donne un second avis. Elle ne dépend ni des concessionnaires, ni du processus de garantie, ni du jugement d'un technicien :

```
with connectee as (
    select cle_vehicule,
           cle_vehicule in (select cle_vehicule from exposees) as dans_la_cohorte
    from marts.fct_sorties group by 1
)
select c.code_anomalie, a.libelle,
       count(distinct c.cle_vehicule) filter (where n.dans_la_cohorte) as cohorte,
       count(distinct c.cle_vehicule) filter (where not n.dans_la_cohorte) as ailleurs
from marts.fct_codes_sortie as c
inner join marts.dim_anomalie as a on a.code_anomalie = c.code_anomalie
```


© 2026 iAlexMG. Tous droits réservés.

```
inner join connectee as n on n.cle_vehicule = c.cle_vehicule
group by 1, 2 order by 1;
```

Quatre-vingt-quinze véhicules de la cohorte sont connectés, contre 7 549 pour le reste du parc.

Le code	Cohorte, sur 95	Ailleurs, sur 7 549
`T310`, surchauffe	12, soit 12,6 %	84, soit 1,1 %
`T320`, liquide bas	17, soit 17,9 %	194, soit 2,6 %
`D240`, courroie	4, soit 4,2 %	496, soit 6,6 %
`E120`, batterie	2, soit 2,1 %	153, soit 2,0 %
`H510`, pression d'huile	1, soit 1,1 %	102, soit 1,4 %

Les deux codes du circuit de refroidissement séparent les groupes d'un facteur dix ; les trois autres ne les séparent pas. C'est ce qui fait la différence entre une corrélation et un mécanisme : la cohorte diverge sur ce qu'un collier mal serré fait diverger, et sur rien d'autre.

Enfin, la courbe dans le temps, qui explique pourquoi le Cours 06 ne pouvait pas conclure :

```
select date_trunc('month', r.cle_date_ouverture)::date as mois, count(*) as reclamations
from marts.fct_reclamations as r
where r.cause = 'REFROIDISSEMENT'
  and r.cle_vehicule in (select cle_vehicule from exposees)
group by 1 order by 1;
```

Une réclamation en décembre, quinze en janvier, quinze en février, cinq en mars. Au 31 décembre, l'entrepôt ne pouvait montrer que la première, et le silence des trois mois précédents.

À retenir — ce n'est toujours pas une preuve, et c'est une bien meilleure raison d'agir. Rien ici n'établit que le compresseur soit la cause : d'autres choses ont pu changer à l'usine entre le 15 et le 28 octobre. Ce que l'entrepôt établit, c'est que 237 véhicules identifiés par leur numéro de série tombent en panne douze fois plus souvent que leurs voisins, sur un circuit précis, et que 29 d'entre eux ont crié avant de tomber. C'est assez pour un rappel ciblé, et c'est ce qu'on demande à un entrepôt.

Étape 19 — La livraison

Complétez `02_Boreal_intermediaire/Solution/02_ae/LIVRAISON.md` : une section pour le sixième processus, avec le grain de `fct_sorties`, ses comptes et son témoin ; une pour `fct_codes_sortie` et `dim_anomalie` ; et la reprise de la section finale, qui annonçait la télémétrie et l'incrémental comme manquants.

Reprenez aussi la ligne qui disait que le fait d'accumulation supporterait mal l'incrémental : elle a maintenant sa mesure, 154 lignes réécrites sur 3 348, et elle explique pourquoi il reste en `table`.

© 2026 iAlexMG. Tous droits réservés.

Récapitulatif

L'état du projet à la fin du cours :

Ce qui est en place	Vérification
Le projet complet	`dbt build --target prod --exclude snp_concessionnaire` rend `PASS=128 WARN=0 ERROR=0`
Les modèles	31, dont 4 arrivés avec ce cours, et 2 incrémentaux
Les tests de données	98, dont 20 arrivés avec ce cours
`stg_sorties`	201 542 sorties, les deux noms de la température réunis
`fct_sorties`	201 542 lignes, 7 644 véhicules, 7 965 681,0 km, du 2023-09-08 au 2026-03-31
`fct_codes_sortie`	1 155 codes sur 1 140 sorties
`dim_anomalie`	5 codes, depuis un seed
Le témoin de la télémétrie	106 636 couples, aucun recul du compteur, 0,032 heure d'écart moyen
Le contrôle tranches contre bloc	201 542 des deux côtés
Les deux environnements	`marts` en production, `dev_marts` ailleurs
L'incident	14,35 pour cent contre 1,16, confirmé par la télémétrie

Le Cours 08 met le projet sous surveillance : un dépôt, une branche, une demande de tirage, et une vérification automatique qui refuse un changement qui casse un test. C'est l'autre automatisation, celle qui garde le code plutôt que la donnée.

Pensez à commiter :

```
git add models tests seeds macros dbt_project.yml
git commit -m "Cours 07 : la telemetrie, l'incremental et la fenetre de reprise"
```

Dépannage

© 2026 iAlexMG. Tous droits réservés.

Le message ou le symptôme	Ce qu'il veut dire	La parade
`Conversion Error: Failed to cast value to numerical` sur un `->>`	un `->>` sans parenthèses avant un `like` ou un `is null` : DuckDB l'a lu dans le mauvais ordre	encadrer l'extraction, `(message ->> 'micrologiciel') like '2.%'`
`Binder Error: ... does not have a column named "charge_le"`	la vue de `STAGING` n'a pas été rebâtie après l'ajout d'une colonne	une vue ne se met pas à jour toute seule : `dbt run --select stg_sorties`
Le fait incrémental a plus de lignes que de messages distincts	`unique_key` manque, ou la fenêtre relit sans clé pour absorber la relecture	remettre la clé, puis `--full-refresh`
Le compte ne bouge pas après la bascule vers `incremental`	c'est le comportement attendu : la table existait, donc le filtre s'applique	`--full-refresh` pour repartir du plein
Une version du snapshot est datée d'aujourd'hui	un `dbt build` nu a lancé l'instantané au présent, hors séquence	garder `--exclude snp_concessionnaire`, et rejouer les quatre lots du Cours 05 dans l'ordre
Les modèles atterrissent dans `dev_marts` alors qu'on voulait la production	`--target prod` a été oublié	la cible ne se devine pas : elle s'écrit à chaque commande
`Compilation Error: ... depends on a node named 'anomalies' which was not found`	le seed n'a pas été chargé	`dbt seed` avant le premier `dbt run` qui le référence
Un compte de la télémétrie diffère de ceux du labo	l'entrepôt ne porte pas les trois mois de 2026, ou pas la génération de référence	vérifier la dernière ligne de `raw.telemetrie_sorties` et relancer le générateur avec `--jusqua 2026-03`
`IO Error: Cannot open file ... utilisé par un autre processus`	la ligne de commande DuckDB est restée ouverte	`.exit` avant toute commande dbt ou tout pipeline

© 2026 iAlexMG. Tous droits réservés.